

Deployment, CI/CD, and DigitalOcean Guide

Tim Glauner
TG Investments and Research, LLC
glaunertim@gmail.com
August 30, 2026



I. PURPOSE

This guide describes a practical production path for the repo using GitHub Actions for CI/CD and a single DigitalOcean droplet for runtime hosting.

II. TARGET ARCHITECTURE

The recommended production layout is:

- 1) GitHub repository for source control and CI/CD,
- 2) one Ubuntu-based DigitalOcean droplet,
- 3) Apache as the public HTTPS reverse proxy,
- 4) one local WSGI process managed by systemd,
- 5) a release directory layout with a `current` symlink, and
- 6) a shared server-side `.env` file that is never committed.

III. FILES ADDED FOR DEPLOYMENT

This repo now contains deployment-supporting assets:

- `wsgi.py`,
- `requirements-production.txt`,
- `deploy/systemd/tgir-quantlib-tools.service.template`,
- `deploy/apache/tgir-quantlib-tools.conf.template`,
- `deploy/env/production.env.example`,
- `.github/workflows/ci.yml`, and
- `.github/workflows/deploy_digitalocean.yml`.

IV. CI PIPELINE

A. Objective

CI should prove that the code installs, compiles, the LaTeX docs build, and the quantitative tests pass before deployment is allowed.

B. Recommended CI Stages

- 1) checkout and Python setup,
- 2) virtual-environment creation,
- 3) dependency installation,
- 4) Python syntax compilation,
- 5) targeted unittest execution,
- 6) LaTeX PDF build for the doc set, and
- 7) artifact upload of generated PDFs.

V. CD PIPELINE

A. One-Time Bootstrap

Before automated deployment, bootstrap the droplet manually:

- 1) create the application root such as `/opt/tgir_quantlib_tools`,
- 2) create a shared `.env` file under `/opt/tgir_quantlib_tools/shared/.env`,
- 3) install Apache and enable proxy-related modules,
- 4) install the systemd service using the template in `deploy/systemd/`,
- 5) install the Apache site configuration using the template in `deploy/apache/`, and
- 6) confirm that `/health` works on the local WSGI bind port and through Apache.

B. Automated Release Flow

The deployment workflow should:

- 1) run after CI succeeds on `main` or manual dispatch,
- 2) upload the repository as a release tarball,
- 3) create a timestamped or commit-hash release directory on the droplet,
- 4) create a fresh `.venv` inside that release,
- 5) install `requirements-production.txt`,
- 6) link the shared `.env` and debug directories into the new release,
- 7) repoint `current` to the new release,
- 8) restart the systemd service, and
- 9) smoke-check `/health`.

VI. DROPLET PREPARATION

A. Packages

On Ubuntu, the typical bootstrap is:

```
sudo apt-get update
sudo apt-get install -y python3 python3-venv
python3-pip apache2
```

B. Apache Modules

Enable the required modules:

```
sudo a2enmod proxy proxy_http headers rewrite
ssl
```

TABLE I
GITHUB ACTIONS DEPLOYMENT SECRETS

Secret	Purpose
DO_HOST	Droplet hostname or IP.
DO_USER	SSH user for deployments.
DO_SSH_KEY	Private key used by GitHub Actions.
DO_APP_ROOT	App root, for example <code>/opt/tgir_quantlib_tools</code> .
DO_HEALTHCHECK_URL	Public health endpoint for final verification.

C. Filesystem Layout

Recommended layout:

```
/opt/tgir_quantlib_tools/
current -> /opt/tgir_quantlib_tools/releases
/<release-id>
releases/
shared/
.env
debug/
```

VII. PRODUCTION CONFIGURATION

A. WSGI Entry Point

Use `wsgi.py` as the production entrypoint so the `systemd` service can run a local WSGI server cleanly.

B. Production Dependencies

Use `requirements-production.txt`. It extends the development requirements with `gunicorn` for the production process manager.

C. Service Template

The `systemd` template starts `gunicorn` on a loopback port such as `127.0.0.1:8008`. Apache proxies public HTTPS requests to that local port.

VIII. GITHUB SECRETS

The deployment workflow assumes these secrets or environment variables:

IX. SUGGESTED RELEASE PROCEDURE

- 1) Open a pull request and let CI pass.
- 2) Merge to `main`.
- 3) Let the deployment workflow publish the new release to the droplet.
- 4) Verify `/health`, login, dashboard rendering, and the cliquet page.
- 5) If needed, roll back by repointing the `current` symlink to the previous release and restarting the service.

X. ROLLBACK

Rollback is intentionally simple:

- 1) list prior release directories,
- 2) repoint `current` to the previous known-good release,
- 3) restart the `systemd` service,
- 4) re-run the local and public health checks.

XI. OPERATIONAL SMOKE CHECKS

After every deploy, run:

```
curl -fsS http://127.0.0.1:8008/health
curl -fsS https://your-hostname.example.com/health
```

Then confirm interactive behavior by logging in and opening:

- `/dashboard`,
- `/trade/equity_cliquet`, and
- `/quantlib-data-model`.

XII. TLS

For HTTPS, use Apache with a real certificate. On a public hostname, `certbot` is the lowest-friction path:

```
sudo apt-get install -y certbot python3-
certbot-apache
sudo certbot --apache -d your-hostname.example
.com
```

XIII. NOTES

This deployment model intentionally stays small and inexpensive:

- one droplet,
- local filesystem storage,
- no database,
- no extra worker host, and
- no paid external platform dependencies beyond the droplet itself.